Sistemas Distribuidos

Jesus Martínez Trejo

Laboratorio S08

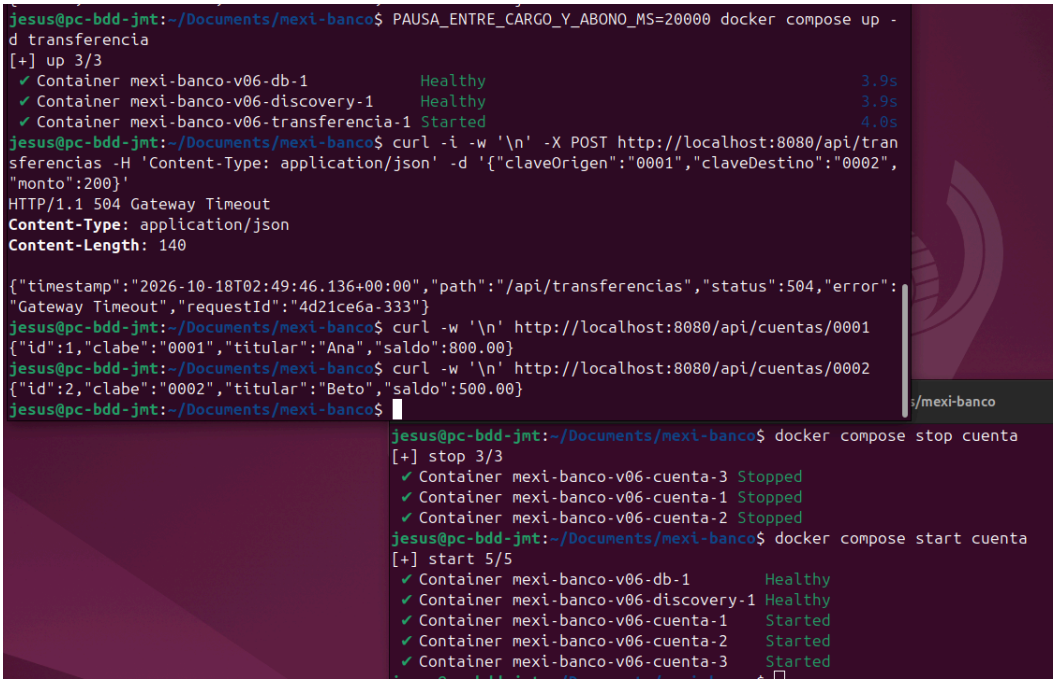
Pasos previos

En primer lugar, se crearon dos cuentas de prueba (0001 para Ana con un saldo inicial de 1000.00 y 0002 para Beto con 500.00) y se consultaron sus saldos iniciales a través del gateway.

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' -X POST http://localhost:8080/api/cuentas
 -H 'Content-Type: application/json' -d '{"clabe":"0001","titular":"Ana","saldoInicial":1000}'
{"id":1,"clabe":"0001","titular":"Ana","saldo":1000}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' -X POST http://localhost:8080/api/cuentas
 -H 'Content-Type: application/json' -d '{"clabe":"0002","titular":"Beto","saldoInicial":500}'
{"id":2,"clabe":"0002","titular":"Beto","saldo":500}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' http://localhost:8080/api/cuentas/0001
{"id":1,"clabe":"0001","titular":"Ana","saldo":1000.00}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' http://localhost:8080/api/cuentas/0002
{"id":2,"clabe":"0002","titular":"Beto","saldo":500.00}
```

A continuación, se levantó el servicio de transferencia configurando un retraso de 20 segundos entre el cargo y el abono (PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000). Después, se inició una transferencia por 200.00 y, durante esa ventana de 20 segundos posteriores al cargo, se detuvieron los contenedores del servicio de cuenta. Al volver a encender cuenta y consultar nuevamente los saldos, se mostró la inconsistencia, ya que los 200.00 se descontaron de la cuenta de origen (0001 bajó a 800.00), pero nunca llegaron a acreditarse en la cuenta de destino (0002 permaneció en 500.00), “desapareciendo” dinero.

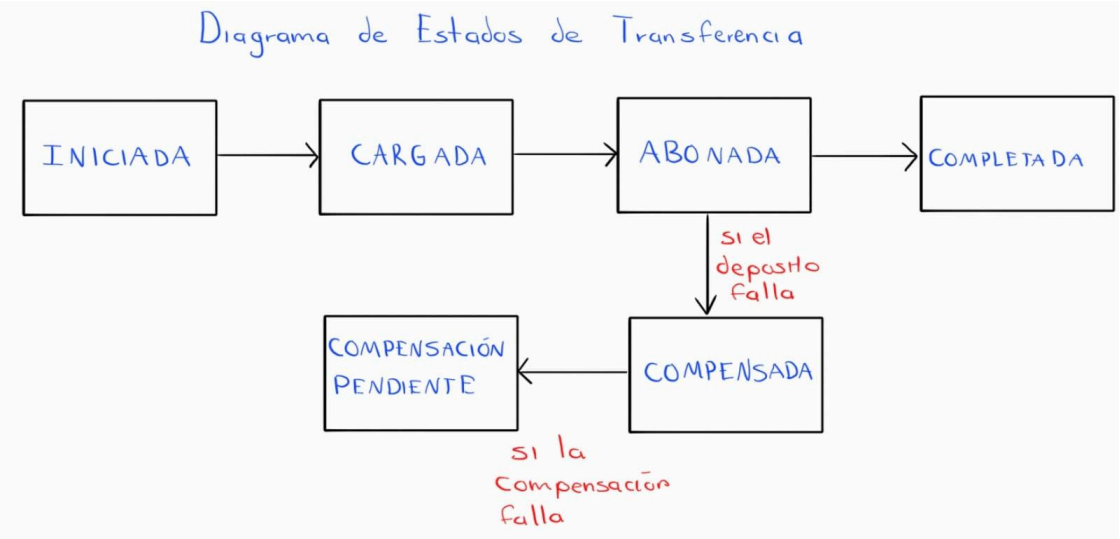
¿Por qué ocurrió esto? Porque al dividir el monolito en microservicios que se comunican por HTTP desapareció la única transacción global, provocando que el servicio cuenta confirmara definitivamente (commit) el cargo en su propia base de datos antes de que fallara la petición remota del abono, sin existir un mecanismo automático que deshiciera el primer paso.



1. Implementación del Patrón Saga

Para implementar el patrón saga en el servicio transferencia, TransferenciaService funge como el orquestador central y ejecuta una secuencia de transacciones locales cuyos pasos y acciones compensatorias se resumen a continuación, seguidos del Diagrama de Estados de Transferencia:

- Paso 1 (Registro inicial – INICIADA): Se registra la transferencia en la base de datos local antes de mover fondos. Si falla, se aborta sin requerir compensación.
- Paso 2 (Cargo al origen – CARGADA): Se descuenta el monto en el microservicio cuenta. Si este paso se confirma y cualquier paso posterior falla, su transacción compensatoria consiste en realizar un abono de devolución por el mismo monto a la cuenta de origen y registrar el movimiento correspondiente.
- Paso 3 (Abono al destino – ABONADA): Se acredita el monto en la cuenta destino. Si falla, dispara la compensación del Paso 2 pasando a COMPENSADA, o bien a COMPENSACION_PENDIENTE si el reintegro también falla.
- Paso 4 (Movimientos y notificaciones – COMPLETADA): Se registran los movimientos finales y se emiten las notificaciones, finalizando la saga.



Para implementar esta lógica en el código, primero se modificó la entidad Transferencia.java añadiendo el enum Estado junto con su atributo de estado, con valor inicial de INICIADA y sus respectivos métodos getter y setter. Esto garantiza que cada transición quede guardada en la base de datos y que, si el servicio se interrumpe a mitad del flujo, la base de datos conserve exactamente el paso en el que se quedó la operación.

```
public enum Estado {
  INICIADA,
  CARGADA,
  ABONADA,
  COMPLETADA,
  COMPENSADA,
  COMPENSACION_PENDIENTE
}
@Enumerated(EnumType.STRING)
@Column (nullable = false)
private Estado estado = Estado.INICIADA;
```

Posteriormente, se implementó la lógica del orquestador en el método transferir() de TransferenciaService.java. Inicialmente, se guarda la entidad en estado INICIADA, enseguida se invoca cuentas.cargar(claveOrigen, monto) y se actualiza el registro a CARGADA. Después, dentro de un bloque try, se incluye una condición que lanza una excepción si la cuenta de destino tiene la CLABE "0000", permitiendo simular un fallo en el abono sin apagar el contenedor de cuenta. Si no hay fallo, se ejecuta cuentas.abonar(claveDestino, monto) y el estado pasa a ABONADA. Finalmente, si el abono lanza cualquier excepción, el flujo entra al catch principal e inicia la transacción compensatoria dentro de un try anidado: invoca cuentas.abonar(claveOrigen, monto) para devolver el dinero al origen, registra en movimiento uno de tipo DEPOSITO por devolución con el saldo restaurado y actualiza el estado a COMPENSADA. Fuera del try-catch interno

se lanza una ReglaDeNegocioException (HTTP 422) explicando al cliente que la operación fue compensada. Si la llamada de devolución dentro del try anidado también falla por caída de cuenta, el catch interno captura el error y se almacena el estado COMPENSACION_PENDIENTE que notifica de una alerta crítica.

```
//1. Inicio de la saga guardando la transferencia
Transferencia transferencia = new Transferencia(claveOrigen, claveDestino, monto);
transferencia = transferencias.save(transferencia);

//2. Se realiza el cargo a la cuenta de origen
CuentaRemota origen = cuentas.cargar(claveOrigen, monto);   // confirmado en cuenta desde aqui
transferencia.setEstado(Transferencia.Estado.CARGADA);
transferencias.save(transferencia);

pausaDeDemo();
CuentaRemota destino;
try{

  //Para pruebas: si el destino es "0000", se lanza una excepcion para simular que el abono falla
  if(claveDestino.equals("0000")){
    throw new RuntimeException("Simulando fallo en el abono hacia la CLABE 0000");
  }

  //3. Se intenta realizar el abono a la cuenta de destino
  destino = cuentas.abonar(claveDestino, monto);
  transferencia.setEstado(Transferencia.Estado.ABONADA);
  transferencias.save(transferencia);
} catch (Exception e) {
  //4. Si el abono falla, se realiza la trasacción compensatoria para devolver el dinero al origen
  try {
    CuentaRemota origenCompensado = cuentas.abonar(claveOrigen, monto);

    //Se registra el movimiento del cargo original y el de la devolución
    movimientos.registrar(claveOrigen, TipoMovimiento.TRANSFERENCIA_ENVIADA, monto.negate(),
      origen.saldo(), "a " + claveDestino + " (fallida)");
    movimientos.registrar(claveOrigen, TipoMovimiento.DEPOSITO, monto,
      origenCompensado.saldo(), "Devolución por fallo en abono a " + claveDestino);

    transferencia.setEstado(Transferencia.Estado.COMPENSADA);
    transferencias.save(transferencia);

  } catch (Exception ex) {
    // 5. Si la compensación también falla
    transferencia.setEstado(Transferencia.Estado.COMPENSACION_PENDIENTE);
    transferencias.save(transferencia);

    throw new ReglaDeNegocioException("Alerta: El abono falló y no nos pudimos comunicar con Cuenta " + claveOrigen +
      " para devolver el dinero. Estado: COMPENSACION PENDIENTE");
  }
  throw new ReglaDeNegocioException("El abono falló ("+ e.getMessage()+
      ") Se devolvió el dinero a la cuenta origen. Estado: COMPENSADA");
}
//6. Si todo sale bien, se registra el movimiento y se notifica a ambos clientes
MovimientoRemoto cargo = movimientos.registrar(origen.clabe(), TipoMovimiento.TRANSFERENCIA_ENVIADA,
  monto.negate(), origen.saldo(), "a " + destino.clabe());
MovimientoRemoto abono = movimientos.registrar(destino.clabe(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
  monto, destino.saldo(), "de " + origen.clabe());

notificaciones.notificar(cargo);
notificaciones.notificar(abono);

transferencia.setEstado(Transferencia.Estado.COMPLETADA);
return transferencias.save(transferencia);
```

Para probar la solución implementada,con el servicio cuenta activo, se solicitó una transferencia de 100.00 desde la cuenta 0001 hacia la cuenta de prueba 0000, lo que automáticamente generó un error en el abono y ejecutó la compensación, donde el servicio respondió con un código HTTP 422 detallando la devolución. Al mismo tiempo, el saldo de Ana (0001) se mantuvo intacto en 800.00 tras el reintegro y se registró el movimiento de devolución con el estado final en la base de datos de COMPENSADA.

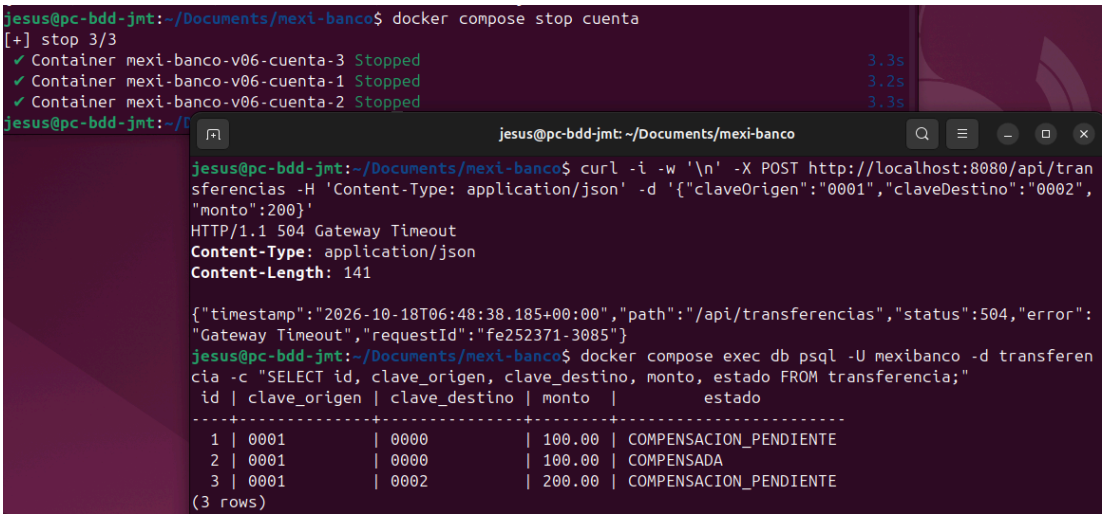
```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' http://localhost:8080/api/cuentas/0001
{"id":1,"clabe":"0001","titular":"Ana","saldo":800.00}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: applicatio
n/json' -d '{"claveOrigen":"0001","claveDestino":"0000","monto":100}'
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 18 Oct 2026 06:06:20 GMT

{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"El abono falló (Simulando fallo en el abono hacia la CLABE
 0000) Se devolvió el dinero a la cuenta origen. Estado: COMPENSADA","instance":"/transferencias"}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -w '\n' http://localhost:8080/api/cuentas/0001
{"id":1,"clabe":"0001","titular":"Ana","saldo":800.00}
```

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ docker compose exec db psql -U mexibanco -d transferencia -c "SELECT id, clave_origen,
 clave_destino, monto, estado FROM transferencia WHERE estado = 'COMPENSADA';"

 id | clave_origen | clave_destino | monto  |   estado
----+--------------+---------------+--------+------------
  2 | 0001         | 0000          | 100.00 | COMPENSADA
(1 row)
```

Por otra parte, al provocar una caída del servicio cuenta durante la ventana de pausa de 20 segundos, fallan tanto el abono al destino como el intento de compensación hacia el origen. Como resultado, la transferencia se almacena en la base de datos con el estado COMPENSACION_PENDIENTE, indicando que otro proceso deberá reintentar revertir el cargo cuando el servicio se recupere.



¿Cómo se completaría la compensación en caso de fallar? Con una tarea o proceso programado que cada ciertos segundos busque en la base de datos todas las transferencias en estado COMPENSACION_PENDIENTE, reintente llamar a cuentas.abonar(claveOrigen, monto) cuando cuenta vuelva a estar en línea y, al lograrlo, actualice el estado a COMPENSADA.

¿Por qué la compensación debe ser idempotente? Porque si el reintento envía el abono de devolución a cuenta, el dinero se acredita en la base de datos, pero la red se corta justo antes de que transferencia reciba la confirmación HTTP, el proceso programado creerá que la petición falló y volverá a enviar la devolución. Sin una llave de idempotencia única por transferencia, el sistema devolvería el dinero dos o más veces al cliente, generando pérdidas para el banco.

2. Implementación de Circuit Breaker

Para evitar que el servicio de transferencia continúe enviando peticiones o quedándose bloqueado cuando el servicio de cuenta está caído, se implementó manualmente un Circuit Breaker en CuentaCliente.java. En esta clase se definió el enumerado EstadoCircuito (CERRADO, ABIERTO, SEMIABIERTO), un umbral de 3 fallas consecutivas y un tiempo de espera de 20 segundos, envolviendo las llamadas HTTP (consultar, cargar y abonar) dentro del método sincronizado ejecutarConCircuitBreaker().

Para probar la solución implementada y verificar la transición entre estados, primero se detuvo el servicio de cuenta y se enviaron tres transferencias consecutivas. Estas tres llamadas fallaron en estado CERRADO, alcanzando el umbral configurado de 3 fallas consecutivas, lo que provocó que el circuito cambiará a estado ABIERTO. Al enviar una cuarta transferencia inmediatamente después, el circuito rechazó la petición al instante sin llamar por red al servicio de cuenta. Posteriormente, se volvió a encender el servicio de cuenta, se esperaron 20 segundos en lo que se pasaba al estado SEMIABIERTO, donde se realizó una nueva transferencia para permitir la llamada de prueba y, al responder exitosamente el servicio de cuenta, regresó al estado CERRADO, como se comprueba en las respuestas HTTP y en los logs del contenedor de transferencia.

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' -d '{"claveOrigen":"0001","claveDestino":"0002","monto":10}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 18:25:07 GMT

{"timestamp":"2026-10-18T18:25:07.063+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' -d '{"claveOrigen":"0001","claveDestino":"0002","monto":10}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 18:25:09 GMT

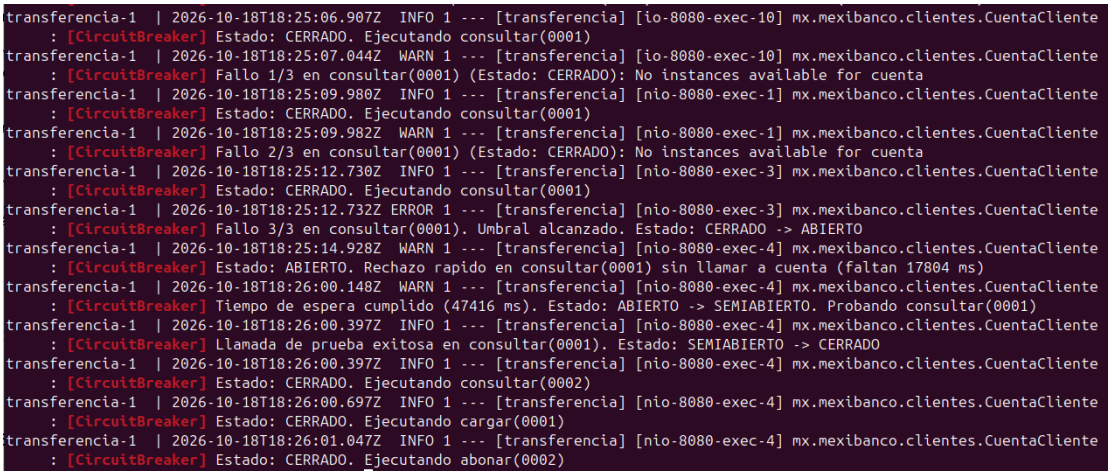
{"timestamp":"2026-10-18T18:25:09.986+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' -d '{"claveOrigen":"0001","claveDestino":"0002","monto":10}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 18:25:12 GMT

{"timestamp":"2026-10-18T18:25:12.736+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' -d '{"claveOrigen":"0001","claveDestino":"0002","monto":10}'
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 18 Oct 2026 18:25:14 GMT

{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Circuit Breaker ABIERTO: el servicio cuenta no est
a disponible temporalmente","instance":"/transferencias"}
```

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ curl -i -w '\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' -d '{"claveOrigen":"0001","claveDestino":"0002","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 18:26:00 GMT

{"estado":"COMPLETADA","id":4,"claveOrigen":"0001","claveDestino":"0002","monto":10,"fecha":"2026-10-18T18:26:00.596764126Z"}
```



¿Qué es lo que la saga no proporciona respecto al aislamiento?

A diferencia de una transacción local ACID en un monolito o del protocolo Two-Phase Commit (2PC), el patrón Saga garantiza consistencia eventual pero no ofrece aislamiento (Isolation) entre los pasos de la operación distribuida. Esto se debe a que el primer paso (cuentas.cargar) confirma de forma inmediata y definitiva el descuento en la base de datos de cuenta, haciendo que los cambios sean visibles para todo el sistema antes de que la saga termine. Por lo tanto, si un usuario u otro servicio consulta el saldo de la cuenta de origen en el intervalo comprendido entre la ejecución del cargo y la aplicación de la transacción compensatoria, observará un saldo temporalmente disminuido, es decir una lectura sucia o estado intermedio no definitivo, aunque el monto finalmente será devuelto cuando el abono falle y se ejecute la compensación. Para mitigar esta falta de aislamiento, algunos sistemas de compras en línea optan por reservar o retener el dinero en el banco y únicamente ejecutan el cobro definitivo una vez que se confirma el envío de los productos.